

Documentação da Estrutura e Fluxo de Execução do Firmware

29 de agosto de 2026

1 Introdução

Este documento descreve a arquitetura principal e o fluxo de execução do firmware do projeto. A lógica apresentada a seguir foi rigorosamente verificada e está em total conformidade com o código-fonte desenvolvido, espelhando precisamente as etapas de inicialização, controle de energia, leitura de sensores e comunicação em rede.

2 Estrutura Principal do Firmware

2.1 Início do Sistema (Boot)

Na inicialização, o sistema executa a função de configuração inicial `setup()`, que realiza as seguintes operações:

- Configura a direção dos pinos do Termopar e inicializa a comunicação SPI e Serial.
- Inicializa o sistema de retenção de dados na memória RTC chamando a função `Backup_Begin()`.
- Inicializa o Sistema de Fornecimento de Energia através da função `SIFE_Setup()`, que avalia o motivo do último reset para ajustar ou reter o cálculo do *State of Charge* (SoC) da bateria.
- Configura os pinos de controle de energia e *sleep* do Modem GSM.
- Inicializa os barramentos de comunicação I2C primário e secundário.
- Configura o *Watchdog Timer* (WDT) do ESP32 para um tempo limite de 2 minutos.

2.2 Loop de Execução Principal (loop)

Durante o ciclo contínuo de operação, o sistema segue o fluxo:

- Reseta o *Watchdog Timer* no início de cada iteração para evitar reinicializações forçadas.
- Chama a função `Gerenciamento_Carga()` para executar a máquina de estados que controla o carregamento da bateria e o *Coulomb Counting*.

- Verifica se é o primeiro ciclo de execução (após 8 segundos de operação) ou se o temporizador atingiu o intervalo de envio padrão definido na variável `TEMPO_ENVIO_AC`.
- Se a condição temporal for verdadeira, executa a rotina `processarLeituraEnvio()` e atualiza o marcador de tempo do ciclo.
- Verifica o estado da variável `fonte`: se estiver operando sem alimentação externa (`FonteOFF`), invoca `iniciarDeepSleep()` para suspender o ESP32 de forma a conservar energia.

3 Bloco de Leitura e Envio de Dados (`processarLeituraEnvio`)

3.1 Coleta e Processamento de Sensores

Nesta etapa, o firmware realiza as leituras físicas e preparações de dados:

- Atualiza a contagem de ciclos chamando `Backup_AtualizarCiclo()` para definir se o ciclo atual exige armazenamento obrigatório na memória RTC em caso de falha.
- Lê a tensão analógica do sensor de pressão via ADS1115 e converte o valor obtido para a medida de pressão real.
- Realiza a leitura de temperatura, aplicando um filtro que coleta 5 amostras, descarta os extremos e retorna o valor central.
- Preenche *buffers* com amostras dos três sensores de aceleração ADXL345 (distribuídos entre dois barramentos I2C) utilizando a função `collectSensorSamples()`.
- Obtém os parâmetros operacionais de energia em tempo real (tensão, corrente, SoC e *flags* de erro) gerados pela biblioteca SIFE.
- Formata os dados coletados de pressão, temperatura e energia em *strings* JSON criptografadas no padrão AES.

3.2 Estabelecimento de Conexão com a Nuvem

A comunicação é estabelecida seguindo estes passos:

- Invoca `conectarRedeEbroker()` para acordar o modem, conectar à rede celular (LTE/GPRS) e estabelecer comunicação com o Broker MQTT.
- Valida a estabilidade da conexão verificando se o IP obtido é válido e se o *socket* MQTT responde.
- Caso seja detectado um IP inválido ou inatividade no *socket*, o firmware força a queda do contexto PDP, aguarda, e realiza uma tentativa de reconexão forçada.

3.3 Tratamento de Falhas de Comunicação

Em caso de falha na rede ou inoperabilidade:

- Se a conexão falhar ou for impossível transmitir os dados, a função invoca `Backup_MarcarFalha()`.
- Executa `Backup_ArmazenarSeNecessario()`, que salva os dados sensoriais e de energia daquele ciclo na memória RTC caso o ciclo seja marcado como importante.
- Desconecta a rede com `desconectarRede()`, coloca o modem em modo *sleep*, e encerra a tentativa de envio.

3.4 Transmissão Bem-Sucedida de Dados

Com a conexão estável, os dados são transmitidos:

- Executa `Backup_EnviarPendentes()` para empacotar e transmitir via MQTT quaisquer registros que estejam represados na memória RTC devido a falhas anteriores.
- Publica o *payload* de energia (`TOPIC_ENERGIA_REAL`), aguardando um curto período de sincronização após o envio.
- Publica o *payload* de pressão (`TOPIC_PRESSAO_REAL`).
- Publica o *payload* de temperatura (`TOPIC_TEMP_REAL`).
- Aciona sequencialmente a função `enviarDadosAcelerometro()` para os três sensores ADXL345, fragmentando os *buffers* extensos e publicando nos tópicos de vibração correspondentes.
- Após concluir as transmissões MQTT, finaliza a etapa chamando `desconectarRede()`.